
Rapport d'activité N°1

Semaine 42 à semaine 46

Rédigé par

CLOÉE HAREAU
STEVEN TRINH
ADRIEN BASCHUNG
WENJIA TANG

1 Travail réalisé

La première partie de notre projet dure 5 semaines, de la semaine 42 à la semaine 46. Nous avons effectué la conception de la grammaire, la mise en œuvre de la grammaire, les applications des tests, ainsi que la correction de notre grammaire.

1.1 La conception de la grammaire

Notre projet commence par la conception de la grammaire. Nous avons premièrement analysé le manuel du langage Tiger ensemble. Pour bien comprendre la structure de la grammaire, nous avons essayé de dessiner les arbres syntaxiques pour l'exemple donné par le manuel en référençant sa partie expression. À la fin de notre première réunion, nous avons décidé de suivre les expressions données par manuel pour avoir une image générale de la grammaire.

1.2 La mise en œuvre de la grammaire

La grammaire a été faite de la semaine 42 à la semaine 43. Pour mettre en œuvre la grammaire, nous avons tout d'abord rédigé les terminaux. Ensuite, chacun a réalisé une partie de la grammaire pour enlever les récursivités gauches locales. De plus, nous avons créé d'autres termes de grammaire pour prioriser les opérations. En même temps, nous avons codé des programmes en langage Tiger pour que nous puissions tester la grammaire après l'avoir fini.

1.3 Les applications des tests et la correction de la grammaire

Les tests ont été appliqués durant la semaine 44. Nous avons trouvé des erreurs syntaxiques et des erreurs de priorité dans notre grammaire. Par exemple, un terme est mal rédigé, ambiguïté du terme de condition et, etc. Par conséquent, nous sommes revenus sur la grammaire pour corriger les erreurs. Nous avons ainsi choisi d'effectuer de simples tests courts pour localiser les erreurs. Finalement, la plupart des erreurs ont été corrigées, cependant, nous avons toujours le problème de priorité d'une affectation.

2 Difficultés rencontrées

Les principales difficultés rencontrées dans l'élaboration de notre grammaire concernent 2 parties de cette dernière : les opérateurs binaires et, dans une moindre mesure, la règle du *if then else*.

2.1 Opérateurs binaires

2.1.1 Récursivité Gauche

Le premier problème auquel nous avons été confronté par rapport à cette grammaire est celui de la récursivité gauche des opérateurs binaires. En effet le manuel nous donne qu'une *expr* peut être dérivée en $expr \rightarrow expr \text{ opérateur_binaire } expr$. Cependant, écrire notre grammaire comme cela l'aurait rendu récursive gauche, on a donc dû modifier cette règle. Nous avons exploré deux solutions :

Première approche

La première a rapidement été abandonnée, elle consistait à appliquer la méthode vu en module de TLA l'année dernière :

Soit une grammaire G Contenant un non-terminal A avec les dérivations suivantes :

$$A \rightarrow \beta_1 | \dots | \beta_m | A\alpha_1 | \dots | A\alpha_n$$

On peut modifier ces règles pour dérécursiver A :

$$A \rightarrow \beta_1 A' | \dots | \beta_n A'$$

$$A' \rightarrow \alpha_1 A' | \dots | \alpha'_A$$

Mais nous n'avons pas réussi à mettre en oeuvre cette solution car dans notre situation, il faut aussi prendre en compte la priorité des opérateurs et la récursivité droite de la règle rend l'utilisation de cette méthode compliquée à combiner avec la priorité des opérateurs, nous avons donc opté pour la deuxième approche.

Seconde approche

La seconde méthode qui est celle qui a été appliquée a été de redéfinir le premier non-terminal accessible depuis l'axiome qui est passé de *expr* à *expr_or* puis cette règle se dérivait en :

$$expr_or \rightarrow expr_and ('|' expr_or)? \text{ (r1)}$$

qui permettait de descendre dans l'arbre pour passer à un opérateur avec une priorité supérieur, tout en ayant la possibilité de mettre plusieurs opérateurs $|$ à la suite. Cette méthode nous a permis dans un premier temps de dérécursiver la grammaire sans la modifier fondamentalement (seuls les non-terminaux *expr* ont du être remplacés par *expr_or*). Enfin tous les non terminaux représentant un opérateur binaires de priorité supérieur ont été gérés de la même façon.

A ce stade il nous restait deux principaux problèmes, l'associativité gauche, et l'affectation qui est de plus basse priorité que *or*, qui n'avait pas encore été traitée car elle était incluse dans la grammaire avant les autres opérateurs binaires.

2.1.2 L'associativité gauche

L'associativité gauche a été notre deuxième problème après avoir réglé la récursivité gauche. En effet, les opérateurs binaires sont (pour la plupart) associatif gauche, mais nous n'avons pas réussi à respecter cette propriété avec la contrainte de non récursivité gauche. La solution précédente produisait une associativité droite. Pour régler ce problème il nous a suffi de remplacer la règle **r1** par :

$$expr_or \longrightarrow expr_and ('|' expr_and) * \textbf{(r2)}$$

ce qui nous a ainsi permis d'afficher toute la suite d'opération sur un même noeuds pour une même priorité d'opération. Il faudra ensuite reformer l'arbre syntaxique de tel sorte qu'il tienne compte de l'associativité gauche lors de l'analyse sémantique.

2.2 if then else

La deuxième partie de la grammaire qui nous a posé problème était la règle :

$$'if' expr_or 'then' expr_or ('else' expr_or)?$$

En effet, une ambiguïté apparaît avec la phrase "*if 1 then if 1 then 1 else 1*" car elle peut être comprise comme "*if 1 then (if 1 then 1) else 1*" ou bien comme "*if 1 then (if 1 then 1 else 1)*". C'est pourquoi nous avons décidé d'ajouter dans la syntaxe de la règle l'obligation de parenthèses après les mots réservés *then* et *else* afin d'explicitier les imbrications et ainsi lever l'ambiguïté. Finalement, nous avons donc la règle suivante :

$$'if' expr_or 'then' '(' expr_or ')' ('else' '(' expr_or ')')?$$

3 Annexes

3.1 Grammaire

```
1 grammar tiger;
2
3 @header{
4 package parser;
5 }
6
7
8
9 program : expr_affect EOF ;
10
11
12 // OPERATIONS
13
14 expr_affect
15     : expr_or (':=' expr_or)?
16     ;
17
18 expr_or
19     : expr_and ('|' expr_and)*
20     ;
21
22 expr_and
23     : expr_test ('&' expr_test)*
24     ;
25
26 expr_test
27     : expr_plus (('='| '<>'| '<'| '>'| '<='| '>=') expr_plus)?
28     ;
29
30 expr_plus
31     : expr_fois (('+'| '-' ) expr_fois)*
32     ;
33
34 expr_fois
35     : expr (('*'| '/') expr)*
36     ;
37
38
39 // "RACINE"
40 expr
41     : '-' expr
42     | 'if' expr_or 'then' '(' seq_expr ')' ('else' '(' seq_expr ')')?
43     | 'let' declaration* 'in' (expr_affect (',' expr_affect)*)? 'end'
44     | 'for' id ':=' expr_or 'to' expr_or 'do' expr_affect
45     | 'while' expr_or 'do' expr_affect
46     | lvalue (call|record|array)?
47     | '(' seq_expr ')'
48     | STR
49     | INT
50     | 'break'
```

```

51     | 'nil'
52     | 'print' '(' expr_or ')'
53     ;
54
55 //liste
56
57 seq_expr
58     : (expr_affect ';' expr_affect)*?
59     ;
60
61 list_expr
62     : (expr_or ',' expr_or)*?
63     ;
64 // DECLARATIONS
65 declaration
66     : type_declaration
67     | var_declaration
68     | fct_declaration
69     ;
70
71
72 // DECLARATION TYPES
73 type_declaration
74     : 'type' type_id '=' type
75     ;
76
77 type
78     : type_id
79     | '{' type_fields? '}'
80     | 'array' 'of' type_id
81     ;
82
83 type_fields
84     : type_field type_fields2
85     ;
86
87 type_fields2
88     : (',' type_field type_fields2)?
89     ;
90     //pour la recursivite gauche
91
92 type_field
93     : id ':' type_id
94     ;
95
96
97 // DECLARATION VARIABLES
98 var_declaration
99     : 'var' id (':' type_id)? ':= ' expr_or
100    ;
101
102
103 // DECLARATION FONCTIONS
104 fct_declaration
105     : 'function' id '(' type_fields? ')' fct2_declaration
106     ;
107
108 fct2_declaration
109     : '=' expr_affect
110     | ':' type_id '=' expr_affect
111     ;

```

```

112
113
114
115
116 // VARIABLES ET AFFECTATIONS
117 lvalue
118     : gen_id(['expr_or'])* (',' id (['expr_or'])*)*
119     ;
120
121 array
122     : '[' expr_or ']' 'of' expr_or
123     ;
124
125 record
126     : '{' id '=' expr_or (',' id '=' expr_or)* '}'
127     ;
128
129 call
130     : '(' list_expr ')'
131     ;
132
133 id
134     : ID
135     ;
136
137 type_id
138     : ID
139     ;
140
141 gen_id
142     : ID
143     ;
144
145
146
147 // Les terminaux (def des exp regulieres reconnaissant les tokens)
148
149 ID
150     : ('a'..'z'|'A'..'Z')('a'..'z'|'A'..'Z'|'0'..'9'|'_'*)
151     ;
152
153 INT
154     : ('0'..'9')+
155     ;
156
157 STR
158     : '"' ('a'..'z'|'A'..'Z'|'0'..'9'|'?'|'!'|'_'|'_'|'.'|':'|';'|','|' '
159     | '\\n'|'('|')')* '"'
160     ;
161
162 WS
163     : [ \n\t\r]+ ->skip
164     ;
165
166 COM
167     : '/*' .*? '*/' -> skip
168     ;

```

3.2 Arbres syntaxiques

3.2.1 Test des opérations

1 - 4 + 5 * - 2 - 4 / 7 | 0

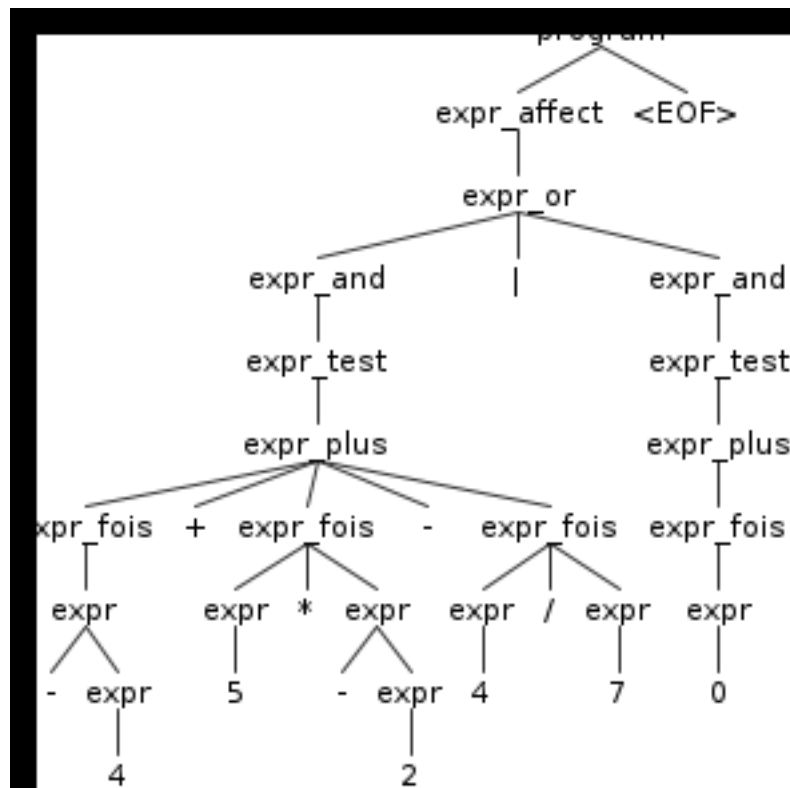


FIGURE 3.1 – Diagramme d'états des cotisations

3.2.2 Test boucle for dans let

```

1 let
2   var a := 0
3 in
4   for i := 0 to 100 do (
5     a := a + 1;
6   )
7 )
8 end

```

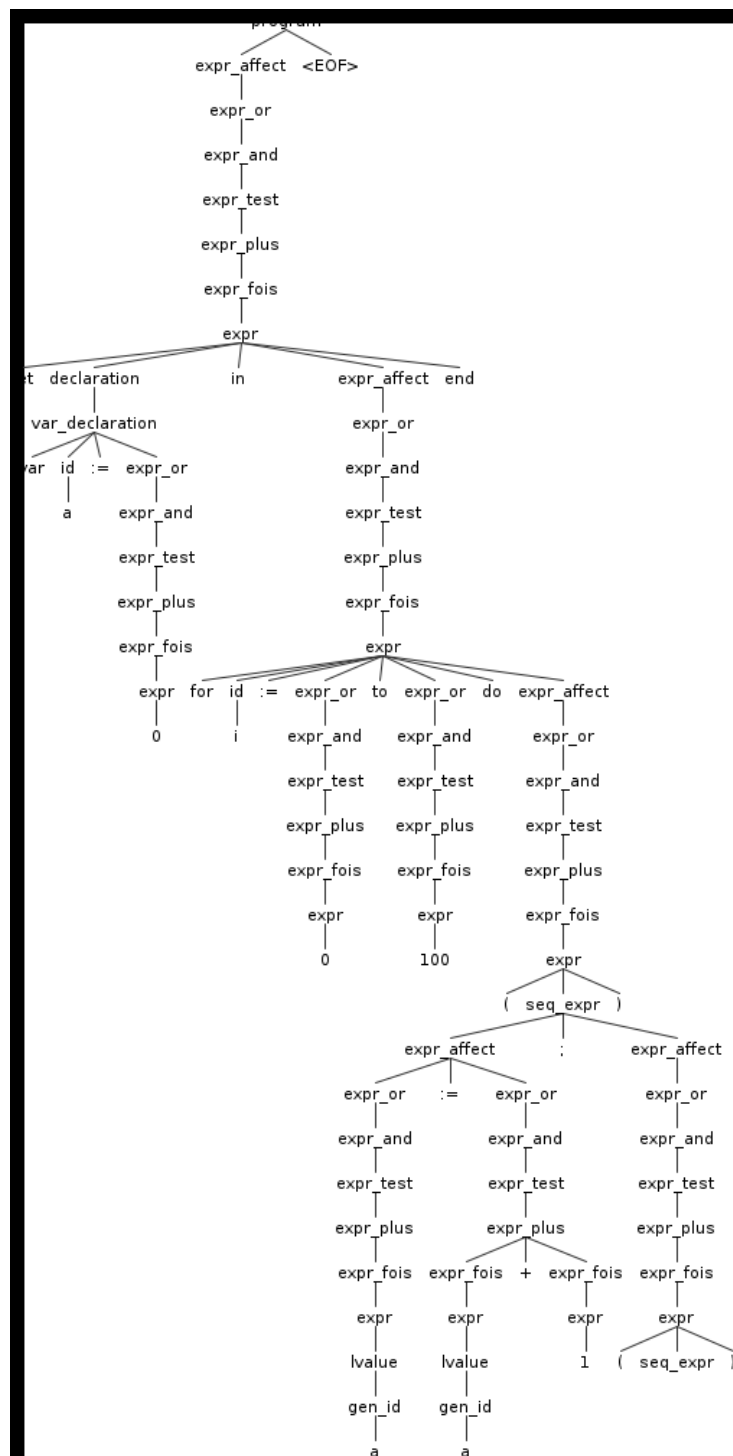


FIGURE 3.2 – Arbre syntaxique boucle for dans let

3.2.3 Test fonction récursive de factorielle

```
1 let
2     var n := 5
3     function fact(k : int) : int =
4         if k = 0 then (1)
5         else (k * fact(k-1))
6 in
7     fact(n)
8 end
```

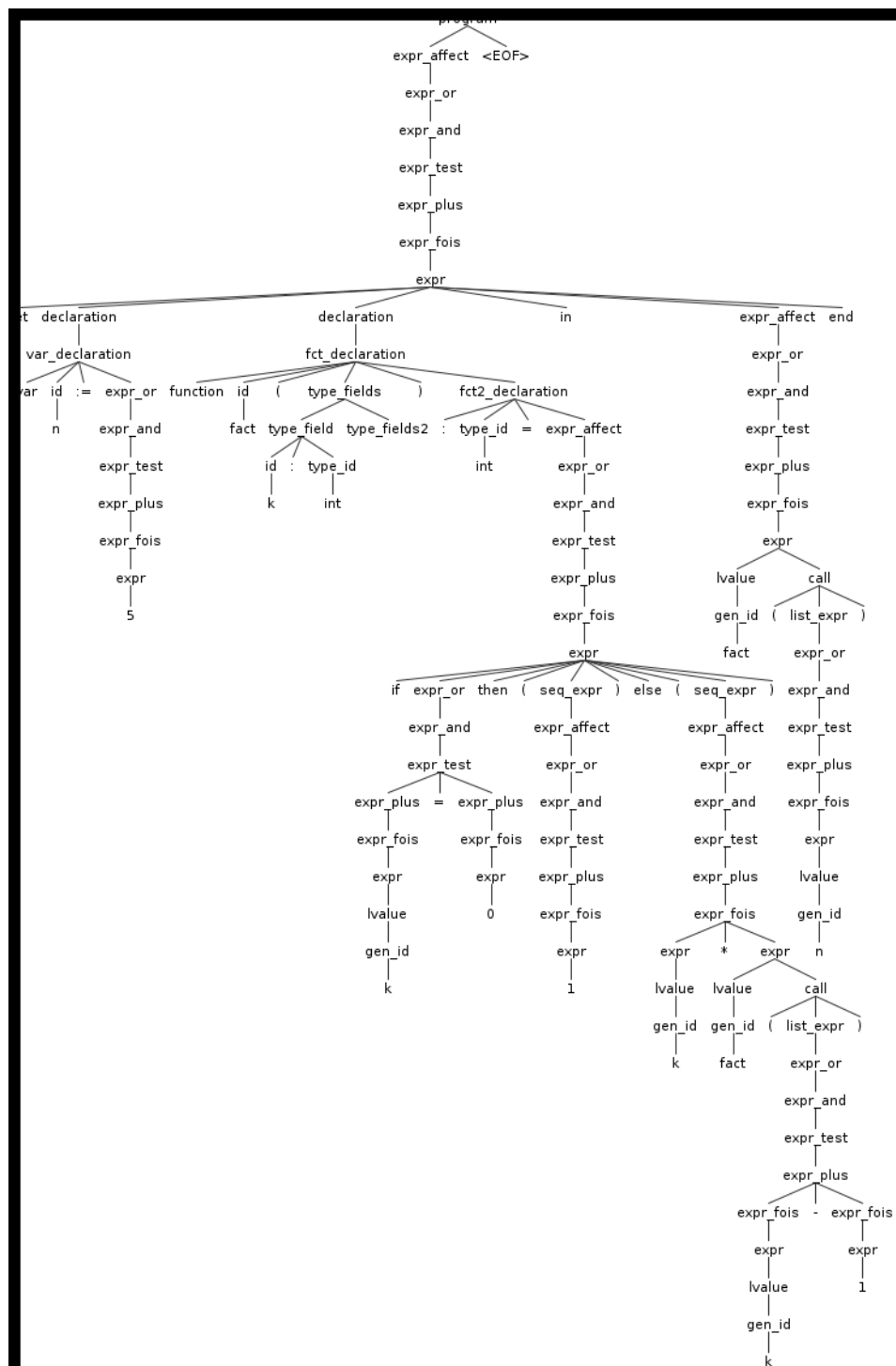


FIGURE 3.3 – Arbre syntaxique fonction récursive de factorielle

3.2.4 Test fonction suite de Fibonacci

```

1  let
2      function fibo(n : int) : int =
3          if n = 0 then (1)
4          else(
5              if n = 1 then(1)
6              else(
7                  fibo(n-1) + fibo(n-2)
8              )
9          )
10     var n := 5
11 in
12     fibo(n)
13 end

```

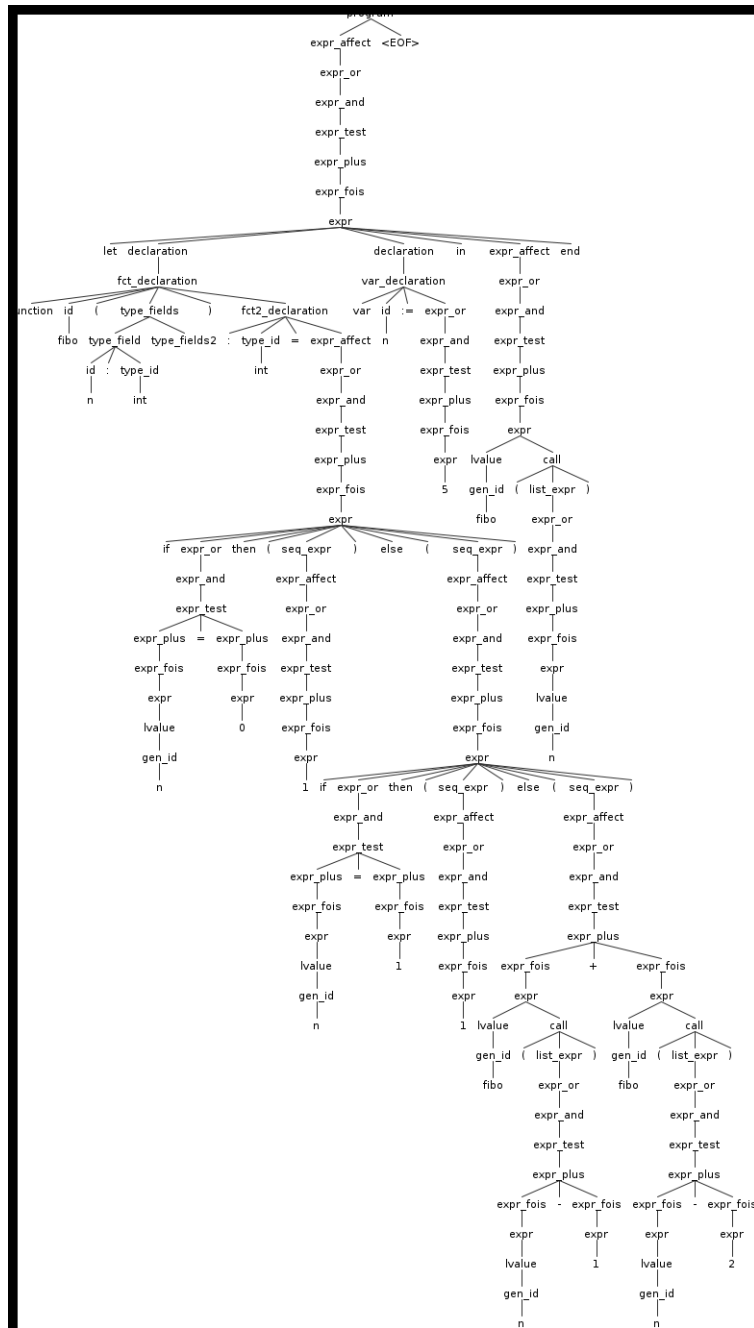


FIGURE 3.4 – Arbre syntaxique fonction suite de Fibonacci